GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-25

今日榜单

1 **makecindy/cindy** TypeScript
Consider it done. The open-source AI agent that works out of the box · 想到，就能做到…
2天

2 **Nutlope/hallmark** CSS
Anti-AI-slop design skill for Claude Code, Cursor, and Codex.
2天

3 **debpalash/VoiceStudio** Python
VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloni...
NEW

4 **pbakaus/impeccable** Rust
The design language that makes your AI harness better at design.
2天

5 **pollen-robotics/microduck** Rust
A Tiny biped duck robot 🦆
2天

6 **TauricResearch/TradingAgents** Python
TradingAgents: Multi-Agents LLM Financial Trading Framework
NEW

7 **Renhuai123/ziwei-doushu** TypeScript
紫微斗数开源排盘引擎 — 基于倪海厦《天纪》体系，含完整排盘算法、四化系统、格局知识…
NEW

8 **JetBrains/thinkrail** TypeScript
Vibe code with pi in a lightweight, real IDE that customises itself around the w...
NEW

9 **coollabsio/coolify** PHP
An open-source, self-hostable PaaS alternative to Vercel, Heroku & Netlify that ...
NEW

10 **cloudflare/security-audit-skill** JavaScript
A coding-agent skill for multi-phase security audits with independently verified...
6天

#1 TypeScript 连续 2 天

cindy

Consider it done. The open-source AI agent that works out of the box · 想到，就能做到。开源、开箱即用的 AI Agent。

Stars **4,516** Forks **636** Today **+677**

https://github.com/makecindy/cindy

项目简介：Cindy 是一个开源、开箱即用的本地 AI Agent 桌面/移动客户端，目标是把 Claude Code、Codex 等多个"执行框架"（harness）与多种模型、工具整合进同一个智能体，让它在你自己电脑的真实文件和已登录应用上完成实际工作。它解决的是当前 AI 编程助手各自为战、切换成本高、难以统一记忆和工具链的问题。

核心功能： - **多 harness × 多模型自由混搭**：Claude Code、Codex 等框架与不同模型可随时切换，甚至同一任务由不同组合并行规划、执行、评审，而工作区、记忆、技能保持连续。 - **跨设备操控**：能驱动浏览器、电脑和手机，并可从 IM 和定时任务中接收工作。 - **可塑的记忆与技能**：纠正一次即可长期生效、跨 harness 共享；工作方式可沉淀为可复用的 Skill。 - **自动化与扩展**：定时任务自运行自汇报，通过 MCP 接入内部系统，插件机制（规划中）可改造 UI 与交互。 - **灵活的模型接入**：支持官方服务、复用已付费的 Claude Code / Codex 订阅、自带 API Key 或本地模型。

适用场景：适合希望把 AI Agent 真正落到日常工作流中的开发者、独立工作者和团队——例如让 Agent 直接操作本地项目代码、自动化重复性任务、通过手机/IM 远程派活，或接入企业内部系统做定制化流程。也适合想审计、fork 或二次开发客户端的研究者。

技术亮点：采用 pnpm monorepo 组织 Electron 桌面端、Expo/React Native 移动端与共享包（认证、设备互联、Agent 编排、模型提供方等）；工具二进制（claude-code、codex、ripgrep、Android platform-tools）按平台在安装时下载并做 sha256 校验，而非入库，保证仓库轻量与可复现；后端服务独立仓库，客户端 Apache-2.0 开源，架构上强调"本地运行 + 真实环境"。

上手建议：先按 README 准备 Node.js 22.x、pnpm 10.x 和 Git LFS，克隆后执行 `git lfs pull && pnpm install`，用 `pnpm restart:desktop:remote --region=cn/global` 启动开发；不想注册可直接选 "Skip Sign-In" 用本地 Agent。想贡献代码请先读 `CONTRIBUTING.en.md`，Linux 用户参考 `docs/linux.md`，插件可经 SkillHub 安装。

#2 CSS 连续 2 天

hallmark

Anti-AI-slop design skill for Claude Code, Cursor, and Codex.

Stars **33,024** Forks **1,716** Today **+1,359**

https://github.com/Nutlope/hallmark

项目简介

Hallmark 是一个面向 Claude Code、Cursor、Codex 等 AI 编程工具的设计「技能包」，目标是让 AI 生成的网页不再带有那种一眼可辨的「AI 味」。它由 Together AI 打造，通过一套硬性设计规则、主题素材库和真实浏览器校验，把 AI 的输出从「模板感」拉回到像人做的设计。

核心功能

- **48 套主题素材库**：每套主题是一份「材质」（纸张、字体、配色、组件、纹理）而非固定模板，AI 会基于它自行排布页面，同一主题下不同需求会产出不同页面。
- **按任务构建页面**：预约页就负责预约、商店页就负责卖货，关键操作放在符合品类惯例的位置，页面以该动作收尾。
- **真实浏览器 lint**：在真实浏览器里检查「机器味」特征——每个标题上方的眉标、字距拉大的大写标签、标题里一个彩色词、满屏空框或纯图片、错位的 hero、每页末尾的邮件订阅、只显示「谢谢」却不真正提交的表单等。
- **重设计与审计命令**：`/hallmark redesign <path>` 保留内容和接线重建页面，`/hallmark audit <path>` 只出报告不改动。
- **接入真实产品**：可指向 URL、dev server 或应用，自动截图并使用 755 张开放授权实拍照片（按主题挑选并在页面署名）。

适用场景

主要面向用 AI 编程工具快速搭原型或落地页的开发者、独立创业者和设计师——当你让 AI 生成页面却总得到千篇一律的「AI 风」时，用它来兜底。也适合需要对现有页面做设计审计或重设计的团队，以及想给 AI 输出加一道质量闸门的工程流程。

技术亮点

最值得关注的是**分级门禁设计**：Floor（对比度、焦点、减弱动效、虚构事实、手机布局崩坏）绝不允许上线；Reflex 是模型容易掉进去的默认套路，有理由可记录在案地推翻；Finish 只是工艺细节、不阻塞发布。再加上「全新上下文的评审者拥有最终话语权」，形成了一套可解释、可申诉的设计质量体系。主题以 ≤8KB 的单文件形式存在，构建时只读取一个，轻量且可组合。

上手建议

先从 `usehallmark.com` 浏览 48 套主题、挑一套符合你产品气质的，然后在 Claude Code / Cursor 中安装该 skill 并让它按你的需求生成页面。想深入的话，读 `skills/hallmark/references/themes/` 下的单个主题文件理解「材质」结构，再尝试 `/hallmark audit` 审计你现有页面；贡献方向可从新增主题或补充 lint 规则入手。

#3 Python NEW

VoiceStudio

VoiceStudio is the open-source, fully-local ElevenLabs alternative — voice cloning, voice design, video dubbing, dictation, transcription & audiobook creation in 646 languages.

Stars **85,904** Forks **9,523** Today **+1,588**

https://github.com/debpalash/VoiceStudio

项目简介：VoiceStudio 是一个完全本地运行的开源语音 AI 工作台，定位为 ElevenLabs 的开源替代品，涵盖语音克隆、语音设计、视频配音、听写、转录和有声书制作，支持 646 种语言。它解决的核心痛点是：用户无需依赖云端付费服务，就能在自己的硬件上完成整套语音生产流程，兼顾隐私与成本。

核心功能：语音克隆（克隆特定人声）与语音设计（用文字描述生成声音）；视频配音（按时间轴对齐语音）；听写与转录（含浮动小组件）；有声书/故事批量生成；以及本地 API 与 MCP 接口供智能体调用。此外还提供"配方"（可复用工作流）和监视文件夹等自动化能力。

适用场景：内容创作者（视频配音、有声书制作）、需要多语言语音的播客/自媒体、注重隐私不愿上传音频的企业与个人、以及希望把语音能力接入自有 Agent 工作流的开发者。

技术亮点：默认引擎基于 k2-fsa/OmniVoice，并支持切换多种引擎，通过"模型管理"界面在本地安装和调度语音模型；硬件层面同时支持 NVIDIA CUDA、Apple Silicon 的 Metal（MPS）加速，以及无独显的集成显卡场景。桌面端已从 Tauri 迁移到 Electron，安装脚本会保留用户设置、项目和模型，工程化程度较高。

上手建议：macOS/Linux 用户可直接用一行 `curl` 安装脚本，Windows 用 PowerShell 的 `irm` 命令，或从 Releases 下载 `.dmg/.exe/.AppImage`。想深入定制或贡献代码，可先阅读 `docs/feature-catalog.md` 了解引擎架构，用 `--main` 参数从源码构建（需 Node.js 22+、Bun、Rust 工具链），并在 Discord 社区交流。注意项目采用 AGPL-3.0 协议，商用集成需留意开源义务。

#4 Rust 连续 2 天

impeccable

The design language that makes your AI harness better at design.

Stars **87,126** Forks **5,204** Today **+1,113**

https://github.com/pbakaus/impeccable

项目简介：Impeccable 是一个面向 AI 编程代理（如 Claude Code）的"设计语言"技能包，目标是让 AI 生成的前端界面摆脱千篇一律的模板感。它解决的核心痛点是：几乎所有模型都基于相同的 SaaS 模板训练，导致产出高度雷同——满屏 Inter 字体、紫蓝渐变、卡片套卡片、彩色背景上放灰字。

核心功能：一是 `/impeccable init` 一次性初始化，把产品定位、受众、约束等"持久事实"写入 `PRODUCT.md`，让后续命令有上下文依据；二是 24 个设计命令（`polish`、`audit`、`critique`、`distill`、`bolder`、`quieter` 等），为 AI 和开发者建立共享的设计词汇；三是 62 条确定性检测规则，CLI 和浏览器扩展可离线运行、无需 API Key；四是 `live/generate` 的浏览器内实时变体迭代，支持直接在页面上生成和挑选设计变体。

适用场景：主要面向用 AI 辅助写前端的开发者、独立产品人和设计工程师，尤其适合"AI 生成的页面看起来都对但都一个样"的情况。典型用法是在项目初期跑 `init` 建立产品上下文，开发中按需调用 `critique/audit` 做审查，上线前用 `polish/harden` 收尾。

技术亮点：引擎用 Rust 写成自包含二进制，首次运行自动下载到 `~/.impeccable/bin/`，技能本身零运行时依赖；确定性规则与 LLM 评审分离——前者无需模型即可跑，保证了可复现性和低成本。从 Anthropic 的 frontend-design skill 起步并显式列出反模式清单（禁用过度使用的字体、禁止纯黑灰、禁止弹性缓动等），思路务实。

上手建议：在项目根目录执行 `npx impeccable install`，然后在 AI 工具内运行 `/impeccable init`，这是最低成本的体验路径。想深入可先读 `impeccable.style` 文档和 Neo Mirai 案例的改造前后对比；贡献方面，确定性检测规则是最容易切入的部分——新增一条"AI 生成设计"的识别规则门槛低、收益直接。

#5 Rust 连续 2 天

microduck

A Tiny biped duck robot 🦆

Stars **11,368** Forks **1,451** Today **+657**

https://github.com/pollen-robotics/microduck

项目简介：Microduck 是 Pollen Robotics 开源的微型双足机器鸭（约 25cm、800g）的"大脑"软件栈——运行在 Rockchip RK3566 上的完整机器人固件。它解决的核心问题是：如何让一个低成本小型机器人，用强化学习策略驱动 15 个舵机实现行走、翻滚、捡物、跌倒自起等复杂动作，并具备可远程更新、可回滚的工程化运维能力。

核心功能：① 50Hz 控制回路，从 ONNX 神经网络策略实时驱动 15 个舵机；② 多模态运动能力：行走、轮式滚动、捡物、跌倒自恢复、坐下、踢球、语音"嘎嘎"叫；③ 完整的 OTA 更新机制，签名验证、健康检查、失败自动回滚；④ 多种交互通道：手柄、蓝牙（duckctl）、手机 App、WebRTC 摄像头推流、深度传感器。

适用场景：机器人爱好者与教育场景（买成品直接玩、用手柄驾驶）；强化学习研究者（配合 microduck_rl 仓库做 sim2real 训练与策略部署）；嵌入式/Rust 开发者（学习多守护进程架构、机器人 OTA 设计）。没有实体机器人的开发者也可用 MuJoCo 仿真环境跑真实守护进程。

技术亮点：纯 Rust 无框架的单 workspace 多守护进程架构（robotd 管控制与电机总线、updaterd 管更新、configd 管网络身份等），统一走 Unix socket 上的 JSON-RPC 契约，所有客户端调用一致；策略训练与部署分离到独立仓库，用 MuJoCo + PPO + 域随机化训练、导出 ONNX 供本仓库加载，是清晰的 sim2real 工程范本；更新机制做到"验证—健康门控—可逆"，这对消费级机器人产品化很有参考价值。

上手建议：有机器人的用户从 `docs/robot/cheatsheet.md`（robotctl 命令速查）开始；想改代码的先读 `docs/design/architecture.md` 理解整体架构，再按 `install-dev.md` 把开发板配置成可接收分支构建；想训练策略则转到 microduck_rl 仓库；手边无硬件可用 `scripts/duck-sim` 在 MuJoCo 中运行真实守护进程。贡献前务必读 CONTRIBUTING.md 了解构建、测试与发布约定。

#6 Python NEW

TradingAgents

TradingAgents: Multi-Agents LLM Financial Trading Framework

Stars **114,043** Forks **22,079** Today **+309**

https://github.com/TauricResearch/TradingAgents

项目简介：TradingAgents 是 Tauric Research 开源的一个基于多智能体大语言模型（Multi-Agents LLM）的金融交易框架，配套 arXiv 论文（2412.20138）。它把"分析师团队协作做投资决策"这件事拆成多个 LLM 智能体分工完成，用来解决单一大模型直接给交易信号时缺乏多视角论证、容易幻觉、难以回溯验证的问题。

核心功能：一是多角色智能体协作，分析师、研究员、交易员、组合经理各司其职，通过辩论/审议形成决策；二是内置回测能力，且强调 point-in-time 数据完整性（只用决策日当天已发布的数据，避免前视偏差）；三是支持多模型分层配置，不同层级（分析师/经理）可跑在不同 LLM 上；四是决策日志与可复现运行（checkpoint 恢复、报告存档为 HTML）；五是丰富的数据源接入（SEC EDGAR、FRED、Alpha Vantage、Polymarket、社交情绪等）。

适用场景：适合量化研究者、金融 AI 方向的学生和研究者做多智能体投研流程的实验与论文复现；也适合希望用 LLM 辅助做基本面/情绪面分析的个人投资者做研究性探索。需要强调的是它更适合研究验证，而非直接实盘交易。

技术亮点：用 LangGraph 编排智能体图结构并支持断点续跑；对"前视偏差"做了系统性处理（各数据源按发布时点过滤），这在 LLM 金融项目里是少见的严谨设计；提供可插拔的 provider registry（OpenAI 兼容端点、Bedrock、Groq、Ollama 等），模型与数据源解耦，工程化程度较高。

上手建议：先读 arXiv 论文理解智能体分工的设计意图，再按 README 配置 API key（支持 `TRADINGAGENTS_*` 环境变量自动探测），从 CLI 用 `--ticker/--date` 跑一次单标的分析看完整流程。想贡献的话，可从 provider 适配、数据源接入或回测正确性测试入手，项目有 CHANGELOG 和 CI 门禁，提交前先对齐版本节奏。

#7 TypeScript NEW

ziwei-doushu

紫微斗数开源排盘引擎 — 基于倪海厦《天纪》体系，含完整排盘算法、四化系统、格局知识库、古籍原文数据

Stars **7,421** Forks **1,521** Today **+489**

https://github.com/Renhuai123/ziwei-doushu

项目简介：这是一个基于倪海厦《天纪》体系的紫微斗数开源排盘引擎，用 TypeScript 实现了完整的排盘算法、四化系统和格局知识库，并附带古籍原文数据。它解决的核心问题是：传统命理排盘长期依赖闭源商业软件，算法不透明、数据不可复用，而该项目把"算法"这一层开放出来，让开发者和研究者可以自由使用、验证和二次开发。

核心功能：一是完整的排盘引擎，涵盖真太阳时校正、中国夏令时处理、农历公历互转、星曜亮度流派切换等细节；二是 13 维解读数据集（v4.0 达 67.5 万张命盘、877 万段解读、解压后 56.6 GB），按年份分文件下载；三是倪师《天纪》原话语料 1848 条，每条可回放到视频具体时间点；四是格局知识库与 405 处可在《紫微斗数全书》中逐字查证的引证。

适用场景：适合三类人——想自建排盘工具的开发者（可直接调用引擎 API）、研究紫微斗数的命理学习者（用数据集做统计或对照验证）、以及做命理类产品的团队（以 MIT 协议复用算法层）。普通用户则可通过作者提供的在线站点体验排盘与 AI 解读。

技术亮点：工程严谨度在同类"玄学项目"中相当罕见——有 5 条排盘回归测试和 GitHub Actions CI，用 5 张金标准盘逐宫核对星曜落宫与亮度，数据集发布前 77 个文件逐个解压逐字节比对。此外把"算法开源、断语库闭源、网站运营自持"的边界划得很清楚，许可分层（代码 MIT / 数据 CC BY 4.0 / 倪师原话非商用）也处理得规范。

上手建议：先看 README 第三章"快速开始"，克隆后 `npm test` 跑通回归测试，再读 `algorithm.ts` 和新增的 `true-solar-time.ts`、`dst-cn.ts` 等模块理解排盘链路。想贡献的话，作者在第八章写明了 issue/PR 口径，可优先从城市经度补全、流派数据校正这类边界明确的任务入手；注意断语库本身不开源，别在 PR 里提交相关内容。

#8 TypeScript NEW

thinkrail

Vibe code with pi in a lightweight, real IDE that customises itself around the way you work — The Vibe You Need

Stars **1,178** Forks **85** Today **+52**

https://github.com/JetBrains/thinkrail

项目简介：ThinkRail 是 JetBrains 孵化器推出的开源"智能体 IDE"，它把 pi 编码智能体包装进一个轻量但完整的开发环境里，核心解决的是"让 AI 编程助手真正理解你的项目并能持续进化"这个问题——不是简单的聊天补全，而是让智能体通过持久化的项目知识和可复用技能，越用越懂你的代码库。

核心功能： - **隔离工作区**：把每个 git 仓库切成独立的 `git worktree`，智能体并行工作时互不干扰，好的分支合并、其余丢弃，完成的工作区会自动"沉降"到搁架，需要时再唤醒。 - **围绕智能体的真 IDE**：Monaco 编辑器、文件树、终端、变更视图一应俱全，支持在文件/差异上留锚定评论并作为结构化上下文发给聊天，也能一键在本地已装 IDE 中打开。 - **Specs 作为事实来源**：代码旁维护一张 SPEC.md 连接图，智能体通过专用 `spec_*` 工具读写维护，让设计意图跨会话存活。 - **多路流式聊天**：每个工作区可并发多个 pi 会话，各自独立模型与 token/成本读数，支持中途转向、排队跟进、委派子智能体，共享计划可被双方编辑。 - **MCP 与扩展**：接入 pi 的 `mcp.json`，对非只读调用先行询问、对仓库自带的服务器默认关闭待批准，Settings 里可逐一开关智能体获得的扩展能力。

适用场景：适合已经在用 Cursor/Claude Code 等 AI 编程工具、但苦于"每次都要重新解释项目背景"的开发者；也适合需要让多个智能体并行处理不同分支任务、并希望把设计意图沉淀为可维护文档的团队。对 JetBrains 生态用户尤其友好。

技术亮点：用 `git worktree` 做物理隔离是相当务实的选择——比容器或虚拟环境轻，又天然契合分支工作流；把 SPEC.md 做成图结构并由智能体专用工具维护，是把"上下文工程"产品化的一种尝试；pi 智能体与 IDE 的深度耦合（流式、成本读数、子智能体）也展示了 agentic IDE 该有的交互密度。

上手建议：先从官网 thinkrail.ai 下载桌面版或用 `thinkrail` CLI 在浏览器中体验，拿一个自己的 git 仓库跑通"开工作区→让智能体改代码→审 diff→开 PR"的闭环；想贡献的话，README 已说明扩展支持是下一步重点，可优先关注 Settings → Extensions 相关的 TypeScript 代码，并留意它作为 JetBrains incubator 项目的开放程度与 issue 区讨论。

#9 PHP NEW

coolify

An open-source, self-hostable PaaS alternative to Vercel, Heroku & Netlify that lets you easily deploy static sites, databases, full-stack applications and 360+ one-click services on your own servers.

Stars **63,932** Forks **5,755** Today **+192**

https://github.com/coollabsio/coolify

项目简介：Coolify 是一个开源自托管的 PaaS 平台，定位为 Vercel、Heroku、Netlify 的替代品，让你在自己的服务器上部署应用、数据库和服务。它解决的核心痛点是：既想享受云平台的部署便利，又不想被厂商锁定、不想支付高昂托管费用、想完全掌控基础设施。

核心功能：一是"任意应用部署"，支持从 GitHub/GitLab/Bitbucket/Gitea 拉取代码，通过 Nixpacks、Dockerfile、Docker Compose 或预构建镜像构建；二是"360+ 一键服务"，数据库等有状态服务带持久化存储和自动生成凭据；三是"自动化部署"，支持 Git push 触发、PR 预览环境、Webhook 和镜像回滚；四是"网络与运维管理"，涵盖自定义域名、自动 HTTPS、反向代理、健康检查、多服务器管理和日志/终端查看。

适用场景：适合有自建服务器（VPS、裸金属、树莓派皆可）的独立开发者、小团队和注重数据主权/隐私的组织，用于替代昂贵的云 PaaS 来托管全栈应用、数据库和内部服务；也适合想低成本搭建类 Heroku 体验的运维人员。

技术亮点：仅需 SSH 连接即可纳管服务器，抽象层做得轻——配置留在用户自己的服务器上，停用 Coolify 后资源仍可独立运行管理，避免了"平台绑架"；采用 PHP 实现但底层深度整合 Docker 生态，通过 Nixpacks/Railpack 实现无需写 Dockerfile 的构建；同时提供 Dashboard、API、CLI、MCP 多种交互方式，MCP 支持意味着可对接 AI 工具链，是较新的设计取向。

上手建议：最快路径是用官方一行脚本 `curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash` 在支持的服务器上安装（建议先阅读 `scripts/install.sh` 再执行）；推荐采用"一台跑 Coolify + 多台跑业务"的分离部署。想深入可先读官方文档的安装指南，贡献代码前务必阅读 `CONTRIBUTING.md`，社区问答在 Discord 和 GitHub Discussions。注意项目靠捐赠维持"永久免费、无付费墙"，若长期使用可考虑赞助。

#10 JavaScript 连续 6 天

security-audit-skill

A coding-agent skill for multi-phase security audits with independently verified, machine-readable findings

Stars **45,998** Forks **2,855** Today **+2,874**

https://github.com/cloudflare/security-audit-skill

项目简介：这是 Cloudflare 开源的一个“编码代理技能”（coding-agent skill），本质是一套结构化的提示词与校验脚本组合，把通用 AI 编码助手改造成一个分阶段执行的安全审计员。它解决的是让 AI 代理做安全审计时容易漏检、误报、结论不可复核的问题，通过多阶段编排和独立验证把审计过程工程化。

核心功能：六阶段审计流水线（侦察 → 覆盖驱动的漏洞挖掘 → 候选验证 → 结构化输出 → 独立记录核验 → 目标无关报告）；用 `coverage-ledger.json` 追踪攻击面覆盖情况，避免遗漏；每个候选漏洞交给全新代理尝试“证伪”，产出 `confirmed/needs_validation/rejected` 三类明确结论；提供零依赖的 schema 校验脚本保证 `findings.json` 机器可读且格式合规；多次运行结果可叠加，复用历史账本定位新缺口。

适用场景：安全团队或开发者想用 AI 代理对自有代码库做系统化漏洞排查时使用；适合需要可追溯、可复核审计记录的合规或内部安全评审场景；也适合作为自建漏洞挖掘流水线的起点（Cloudflare 自己的 harness 就由此演化而来）。

技术亮点：把审计拆成“隔离猎手 + 覆盖批评者 + 独立验证者”的多代理协作模式，用角色隔离降低单一代理的确认偏误；攻击面按领域拆成大量专门文档（内存安全、LLM 提示注入、Web 协议、供应链、云部署等），实现按目标类型加载对应狩猎类别；结论分级严格——`needs_validation` 不允许带严重性，`confirmed` 必须有完整溯源，从机制上压制“看起来像漏洞”的误报。

上手建议：先用 `npx skills add` 安装技能，然后在目标代码库中直接对代理说“security audit this codebase”跑一次完整流程，观察产出的 `architecture.md`、`coverage-ledger.json` 和 `findings.json`。想深入理解或贡献，建议从 `SKILL.md` 读核心原则与反模式，再看 `VALIDATION-AND-REPORTING.md` 理解验证逻辑，脚本部分可从 `validate-findings.cjs` 及其测试入手。

数据来源：github.com/trending

报告日期：2026-10-25

由 DeepSeek AI 提供智能分析 | 自动生成